

TrustBank CBS — Production-Gate Remediation Plan

Now ~77/100 · from 62 → shippable ~74 · Companion to [ARCHITECTURE-REVIEW.md](#) Plan date: 2026-07-10 · Status (updated 2026-07-31): IN PROGRESS — score 62 → ~77; 6 gate items full + H-2/H-6 partial, plus Observability, #12 DB migration tooling (both 80-band) and the controller/browser test tiers + M-4 PII masking + M-3 login rate limiting (90-band) done early. Numerically past the gate, but NOT yet green — needs H-6 rollout to the remaining write services + H-3 (git + CI).

🔄 **Progress.** ✅ **C-1** (auth-bypass fail-closed guard) · ✅ **C-2** (PBKDF2 passwords end-to-end: primitive, all 1,731 users migrated, login verify, change-password screen; the A5 legacy-column cleanup is held) · ✅ **H-1** (global antiforgery in `AddCbsFramework` + AJAX token header) · ✅ **H-4** (guard-ordering startup assertion + 2 pinned tests) · ✅ **M-1 / security headers** (fully strict CSP — `script-src / style-src 'self'`, no nonce, no `unsafe-inline` — + `X-Frame-Options/nosniff/Referrer-Policy/Permissions-Policy`; all inline scripts/handlers/styles refactored out) · ✅ **Controller + E2E tests** (90-band, done early — `...UI.IntegrationTests` + `Tf1Cbs.E2E`; Testability 5→7) · ✅ **M-4 / PII masking on every host** (90-band, done early — canonical `logmasking.json` linked into all five hosts, `optional: false` before the logging pipeline; the thin hosts had been **half-masked**, regex redaction silently off; `LogMaskingConfigTests`, mutation-verified) · ✅ **M-3 / login rate limiting on every host** (90-band, done early — `cbs-login` policy in `AddCbsFramework`, `UseRateLimiter()` first in `UseCbsFramework`, both credential POSTs, per client IP + path, 429 + `Retry-After`; the gateway's limiter was *volumetric* and the single-host shapes had none. Its prerequisite closed with it: `ForwardedHeaders` gained `CIDR KnownNetworks` so the Docker multi-host stack actually trusts the gateway, malformed trust entries now fail startup, and a *missing* one is warned about once per host start; 25 tests) · ✅ **Observability** (80-band, done early — `AddCbsObservability`: OTel traces+metrics+OTLP+DAL spans+log-trace correlation, wired in all 5 hosts + gateway, off by default) · ⚠️ **H-2** (secrets out of committed config; `sa` rotation + least-priv login remain external). **Remaining for a green gate:** H-3 (git + CI), H-6 (optimistic concurrency — ⌚ **pattern established on the State master** as the reference implementation, using **Tf1OmniDb's native row-version** (`[DbColumn(IsRowVersion=true)]`) → repository auto-increments + guards + throws `ConcurrencyException`), DB-verified by a passing conflict test; rollout to the other write services deferred by decision). Done items are marked ✅ in the tables below.

0. At a glance — what's left

Orientation only. \$5 sequences the gate items, \$9 is the authoritative road-to-100 table with owners, bands and effort; when this list and those tables disagree, the tables win.

Blocking a green gate — the score is numerically past ~74, these are why it is still amber:

- **H-3 — git init + CI.** The working tree is still not a repository, so nothing runs the ~300 tests automatically.
- **H-6 — optimistic concurrency rollout.** Proven on `StateService`; **every other write service is still last-writer-wins.** Three mechanical steps per service.
- **H-2 finish** — rotate `sa`, least-privilege SQL login, real TLS cert. DBA/ops, not code.

Known open defects (found by the E2E tier, pinned with `test.fail()` — see risk rows 21–24):

- A login *attempt* on a dormant account **deactivates it**, with no recovery short of a DBA `UPDATE` .
- Signed cookie + closed session $\Rightarrow$ **infinite** `/Account/Login` $\rightleftharpoons$ `/Modules` **redirect loop**; the user cannot reach the form to recover.
- `cbs-account-picker.js` undoes `Model.Disabled` at runtime (latent — no current consumer takes that branch).
- The component showcase wires `clearTargets` backwards, so the reference demo teaches the wrong thing.

~80 band — Redis session + gateway affinity · observability **backend** (dashboards/SLO alerting; instrumentation is done) · DB-side paging (blocked on a `B_ACCOUNT` index — DBA, not coding) · reference-caching rollout (deferred by decision) · Polly retry (backlogged on write-idempotency rules) · DbMigrator `0000` baseline + a live PostgreSQL run.

~90 band — IaC/Kubernetes · DR/HA · container hardening (non-root) · WAF · pen test · threat model · row-token bypass tests + multi-host SSO browser flow · **M-5** (`Database:LogParameterValues` can still log PII outside Dev) · HSTS strengthening · the IIS `KnownProxies` step (now detectable, not enforced).

The actual mountain — migration completeness: ~98% of screens remain. 12 domain screens / 47 views across 8 domains are done. Worth **+9** alone; pure volume, no redesign.

If you do only two things next: H-3 and H-6. They are the only remaining *gate* items that are code/infra you control, they are cheap against their value, and they parallelise across people. Everything above them is ops procurement or the migration grind. **Worth stating plainly:** with M-3 and M-4 closed, **no application-code row remains open under Security** — what is left in that category is process (threat model, pen test, WAF) and ops (sa rotation, HSTS). That is the milestone the long finding list tends to obscure.

Decisions this plan is built on

Three scoping decisions were made before drafting:

1. **Target = clear the production gate.** Fix the Critical + High blockers only; do *not* chase the Medium tier (observability, distributed session, DB-migration tooling, DIP extraction). Those belong to the "80+" band and are listed at the end as the next program, not this one.
2. **Break bug-for-bug legacy parity where parity is the root cause.** Specifically, replace reversible password storage with a real one-way scheme, built on the in-house `Tf1SecurityCrypto` library (`_Archive\Ad.ProjectLibrary\Tssipl.Security.Cryptography`) rather than a new third-party dependency.
3. **This is a plan document.** It sequences the work, estimates effort, and defines "done" per item. It makes no code changes.

The gate in one line: eight fixes across four workstreams — an immediate auth-bypass patch, real password hashing, CSRF, secrets externalization, security headers, a guard-ordering assertion, optimistic concurrency, and the source-control/CI safety net that keeps them from regressing. Clear these and the system moves from "*fails an audit on day one*" to "*defensible to deploy*."

1. What "the gate" actually is

Nothing here is a redesign. Every item is a bounded, well-understood change to a codebase whose *structure* already scores 8/10. The reason the overall sits at 62 is that a handful of items act as a **hard ceiling**: as long as a config flag can bypass authentication and passwords are recoverable, no amount of clean architecture makes the system shippable. Removing the ceiling is what this plan buys.

#	Blocker	Sev	Root cause	Fix class
C-1 ✓	MFA_AD_LOGIN_ENABLED auth bypass	Critical	Faithful port of the legacy proc whose upstream AD authenticator was never migrated	✓ DONE — fail-closed guard + pinned test
C-2 ✓	Reversible password storage	Critical	3DES/MD5 via TflSecurityCrypto — recoverable at rest	✓ DONE — PBKDF2 primitive + full migration + login/change-password verify
H-1 ✓	No CSRF on ~30 POST actions	High	AddControllersWithViews() without the antiforgery filter	✓ DONE — global antiforgery in AddCbsFramework + AJAX token header
H-2 ⚠	Committed sa prod creds + TrustServerCertificate=true	High	Connection strings in base appsettings.json	⚠ MOSTLY — externalized; rotate sa + least-priv login + real cert pending
H-3	No source control / CI / IaC	High	Working tree is not a git repo	Init repo + pipeline
H-4 ✓	Latent fail-open on guard ordering	High (latent)	GetEndpoint()==null → next() with no assertion	✓ DONE — EnsureEndpointRoutingRegistered() startup assertion + 2 pinned tests
H-6 ⌚	No optimistic concurrency	High	Load → mutate → rewrite-all-columns; last-writer-wins	⌚ PATTERN ON STATE — TflOmniDb-native row-version ([DbColumn(IsRowVersion=true)] → auto-increment + WHERE guard + ConcurrencyException ; conflict test passes on dev DB); rollout to other write services pending
M-1 ✓	No security headers	Medium*	No header middleware	✓ DONE — UseCbsSecurityHeaders (fully strict CSP + 4 companion headers), inline scripts/handlers/styles refactored out

* Security headers are formally a Medium, but they are cheap and materially lift the Security category, so they ride along in this gate.

+ **C-1 is split** (decided 2026-07-10). The *gate* item is the **fail-closed guard** — AuthenticateUserAsync must never return success on an unverified credential; when a login-mode flag is on but its authenticator isn't implemented, it returns Result.Fail . That closes the bypass in a same-day change. The **AD / QR / OTP login modes themselves are confirmed roadmap features, not part of this gate** — they are tracked in [BACKLOG.md](#) ("Authentication & login

modes") and listed under §8. This split matters because AD login *is* wanted, so the fix is **not** "delete the branch" (that would preempt the future AD path) — it's "never trust an unverified credential until a real authenticator says so."

2. How 62 becomes ~74 (currently at ~77)

The overall is a holistic judgment, not a column average — but the category deltas show where the lift comes from. Only categories the gate touches move; everything else holds.

Category	Now	After gate	What moves it
Security	3	7	Auth bypass closed, passwords hashed, CSRF, secrets externalized, headers. (Not 8–9: no full pen-test, WAF, or threat model yet.) <i>(Two 90-band rows were also collected early — M-4 PII masking and M-3 login rate limiting — so the application-code side of Security is now clear; everything still open in this category is process (pen test, threat model, WAF) or ops (sa rotation, HSTS). The cell stays at 7 for exactly that reason.)</i>
Deployment Readiness	5	7	Git + CI with test/secret/SAST gates. (Not higher: no IaC/K8s.)
Database Design	5	6	Optimistic concurrency on writes. <i>(Reached 6 out of band on 2026-07-30 via #12 migration tooling instead; caps at 6 until the H-6 rollout AND a 0000 legacy-schema baseline both land.)</i>
Testability	5	6	CI actually runs the suite; new auth/concurrency tests. <i>(Overtaken 2026-07-30: reached 7 out of band — past this gate target — because the controller/HTTP tier ...UI.IntegrationTests and the browser tier TflCbs.E2E both shipped, clearing the "no controller/E2E" dock the plan had assumed would still be open at the gate. Held below 8 by the unmockable sealed DataAccess, the live-DB dependency in every authenticated tier, and H-3 still not running any of it — so the "CI actually runs the suite" half of this row is still outstanding.)</i>
Structure, Maintainability, Extensibility, Code Quality, Documentation	8/7/8/8/9	<i>unchanged</i>	Already strong; gate doesn't touch them.
Scalability, Performance, Observability, API, Dependency Mgmt	4/5/ 7 /6/6	<i>Observability 4→7 (instrumented, done early); rest unchanged</i>	Next band (80+); observability instrumentation banked ahead of gate.

Raw category sum moves **83 → 91 / 140**. The *holistic* number moves further — to a **72–75 band** — because the fixed items were suppressing the whole score as ship-blockers, not just their own cells. **Planning figure: ~74, and (more importantly) the gate flips from red to green.**

3. Workstreams & sequence

Four workstreams. The ordering is deliberate: the safety net goes in *first* so every subsequent fix lands behind a test/secret gate, then the same-day security patches, then the two changes that need care (passwords, concurrency).

Phase 0	Safety net	— git + CI —————	(enables all below)
Phase 1	Same-day security	— C-1 · H-4 · headers · H-2 secrets —	(small, high-confidence)
Phase 2	Auth hardening	— C-2 passwords · H-1 CSRF · login RL —	(needs migration care)
Phase 3	Data integrity	— H-6 optimistic concurrency ————	(schema + pattern change)

Phase 0 — Source control + CI (H-3) — the prerequisite

- **git init** (a correct `.gitignore` already exists — it excludes `.env`, `certs/`, `.pfx`, `bin/obj`). *First commit is the current tree* after* H-2's secret sweep, so no secret ever enters history.
- **Pipeline** (GitHub Actions or Azure DevOps): `build Tf1Cbs.Entities` → `build solution` → `dotnet test Tf1Cbs.Tests` + `Tf1Cbs.ArchTests` → **secret scan** (gitleaks) → **SAST** (`dotnet analyzers` / CodeQL). Red blocks merge.
- **Note on tests:** the suite currently needs a live DB and collapses to ~0 cases without one. CI value on day one is the failure-path facts + arch tests + build + scans. A containerized SQL Server service in CI (to make the cross-provider theories run) is a fast-follow, not a gate blocker.
- **Effort:** Medium · **Risk:** low · **Done when:** a PR with a planted secret or a failing arch test is blocked by CI.

Phase 1 — Same-day security

- **C-1 auth bypass — fail-closed guard.** At [AuthenticationService.cs:299](#), the `&& mfaAdLoginEnabled == 0` short-circuit currently lets a non-zero flag skip the password compare with nothing in its place. **Decided (2026-07-10): AD login is on the roadmap, so do not delete the branch — that would preempt the future AD path. Instead make it fail-closed: `AuthenticateUserAsync` never returns success on an unverified credential; when `MFA_AD_LOGIN_ENABLED` (or the QR/OTP flags) is on but no authenticator is implemented, return `Result.Fail` (e.g. "AD login is enabled but not yet available"). When AD/QR/OTP are later built (backlog, §8), the real authenticator replaces the stub. Invariant to pin with a test: wrong password + flag on ⇒ login fails.** **Effort:** Small · **Risk:** low (behind a flag today).
- **H-4 guard-ordering assertion** — add a startup check that routing middleware precedes `UseCbsFramework()`, or make `CbsAccessMiddleware` fail-closed when `GetEndpoint()` is unexpectedly null on a routed path. **Effort:** Small.
- **Security headers** — one middleware adding CSP, `X-Frame-Options: DENY`, `X-Content-Type-Options: nosniff`, `Referrer-Policy`, `Permissions-Policy`. CSP needs a nonce for the one JSON `<script>` island in `_Layout.cshtml`. **Effort:** Small–Medium (CSP tuning is the only real work).
- **H-2 secrets** — move every connection string out of committed `appsettings.json` into user-secrets (dev) / environment or a secret store (prod); remove `TrustServerCertificate=true` (issue a real SQL TLS cert); replace `sa` with a **least-privilege** application login. Docker already externalizes via `${...}` — extend that discipline to IIS/direct hosts. **Effort:** Small–Medium · **Ordering:** must precede Phase 0's first commit.

Phase 2 — Auth hardening

- **C-2 passwords** — the centerpiece; detailed in §4.
- **H-1 CSRF** — register `AutoValidateAntiforgeryToken` globally; emit `@Html.AntiForgeryToken()` in forms; for the `fetch`-based search/JSON endpoints (`cbs-search.js`), send the token in a request header and configure the antiforgery header name. The single-use `RowToken` already covers record-scoped writes, but `Save/Login/Logout` need the token. **Effort:** Small-Medium · **Watch:** the command-dispatch POSTs and `formation` routes all funnel through standard MVC actions, so the global filter covers them uniformly.
- **✓ DONE (2026-07-31) — Login rate limiting.** A login-specific limiter now runs on *all* hosts, not just the gateway. `AddCbsFramework` registers the `cbs-login` policy (fixed window, partitioned by **client IP + request path**, default 10/60 s via `Security:LoginRateLimit`) and `UseCbsFramework` inserts `UseRateLimiter()` **first**, so a throttled attempt is refused before session load, cookie decryption or any DB round-trip; `/Account/Login` and `/Account/ChangePassword` opt in with `[EnableRateLimiting]` . Because `AccountController` ships in `Tf1Cbs.Core.Authentication.Web` (an application part in every host), one attribute covers all 5 hosts and every deployment shape — including the gateway-less single-host one. Rejections are **429 + Retry-After** , logged under the `Security` category. As planned, it is **paired with, not a replacement for, the DB lockout policy**: the limiter is the cheap volume control, `MAXIMUMLOGINATTEMPT` / `REMOTEHOSTLOGINATTEMPT` remain the per-account/per-IP lockout (the per-username dimension can't move into middleware — partitioning on the posted username needs a synchronous form-body read, which Kestrel disables by default). 9 tests in `LoginRateLimitTests` . **Prerequisite for multi-host — closed the same day:** the partition is per real client only where `ForwardedHeaders` trusts the gateway, and the Docker multi-host stack shipped with trust commented out (the framework read exact IPs only; compose IPs are dynamic). `UseCbsForwardedHeaders` now also takes `ForwardedHeaders:KnownNetworks` (CIDR), `compose.multi.yaml` trusts the bridge pool, `compose.multi.https.yaml` tightens back to the static gateway IP, a malformed trust entry fails startup instead of being skipped, and a *missing* one is caught at runtime — a discarded `X-Forwarded-For` logs one `Security` warning per host start, since that misconfiguration otherwise throws nothing and breaks no page. 16 tests (`ForwardedProxyTrustTests` + `UntrustedForwardedHeaderWarningTests`). IIS farms still pin `KnownProxies` by hand — now detectable rather than silent, though not enforced (the deploy script prints the reminder and what breaks without it).

Phase 3 — Data integrity

- **H-6 optimistic concurrency** — add a rowversion/concurrency token to the entities on write paths; change the update pattern from "rewrite all columns" to a guarded `UPDATE ... WHERE key AND version=@original` ; surface a lost-update as `Result.Fail("record changed since you loaded it")` . **Bake this into the money-movement path from the start** — it currently lives in the excluded `Lab` project, so it can adopt the pattern before promotion rather than being retrofitted. Also **verify the X_SCROLL unique constraint** that the existing `MAX+1` retry depends on actually exists in every target DB. **Effort:** Medium · **Risk:** medium (touches schema + every write service).

4. The password fix, in detail (C-2)

This is the item the parity decision was about, so it gets its own section — and it needs an honest read of what `Tf1SecurityCrypto` gives you today.

Current state

The login compares the submitted password, run through `Tf1SecurityCrypto.Cryptography`, against a stored value ([AuthenticationService.cs:296-299](#)). The library's usable password path is **3DES-ECB** (`Encrypt(text, useHashing:true)`, [Cryptography.cs:27](#)) or raw **MD5** (`Encrypt(text)`) — both recoverable/inadequate. A DB compromise yields plaintext.

What the library already offers — and its two gaps

`Tf1SecurityCrypto` **does** ship a salted-hash primitive: `Cryptography.Encrypt(text, salt)` and `Cryptography.VerifyHash(text, salt)`, backed by `SimpleHash.ComputeHash(..., "SHA256", ...)`. Using it would already be a **large improvement** — one-way and salted instead of reversible. But for a bank it has two real gaps you must not gloss over:

1. **It's single-pass SHA256, not an adaptive hash.** No key stretching → fast to brute-force offline on GPUs. The random-salt path also uses only a **4–7 byte salt** (`SimpleHash.cs:27-32`) and a **hard-coded pepper** (`"~!@#$$%^&*()_+|"`) baked into source — a static, non-secret value.
2. **The public `VerifyHash(text, salt)` can't verify against a stored hash.** Its signature takes no stored value; it recomputes a hash from `text` and compares to *itself* (`Cryptography.cs:129-135`) — effectively always true for the same input. It is unusable as-is for login verification.

Recommended: add a proper primitive to `Tf1SecurityCrypto`

This honors "use `Tf1SecurityCrypto`" while actually fixing the root cause — keep it in the library the team trusts, add **no external package**, use only in-box .NET crypto:

- New `PasswordHash.Create(password) / PasswordHash.Verify(password, stored)` using **PBKDF2-HMAC-SHA256** (`Rfc2898DeriveBytes`, in-box, FIPS-friendly), a **16-byte per-user random salt, ≥100,000 iterations** (tune to ~100 ms), and a **versioned self-describing format** — e.g. `v1$iterations$base64(salt)$base64(hash)` — so iteration counts can rise over time without breaking stored values.
- Give `Verify` a real `(password, storedHash)` signature that parses the format, re-derives with the embedded salt/iterations, and does a **constant-time** compare (fixes the `!=` timing side-channel too).
- *(Alternative if the library truly cannot change: PBKDF2 via `Rfc2898DeriveBytes` directly in `AuthenticationService` — same crypto, also zero new dependencies. The plain salted-SHA256 helper is the floor, acceptable only as a stopgap with the caveats above stated in writing.)*

Migration — no big bang

The login path is a single choke point (`AuthenticateUserAsync`), which makes a **rehash-on-login** migration clean:

1. Add a `PASSWORDVERSION` (and, cleanest, a new `PASSWORDHASH`) column to `A_USER` so old and new coexist.
2. On login: if the stored value is new-format → `PasswordHash.Verify`. If legacy → verify with the old scheme; **on success, immediately re-hash with `PasswordHash.Create` and store**, stamping the new version.
3. Users migrate transparently as they log in. Dormant accounts get a **forced reset** after a cutover window (also closes the "change-password screen is a dead `href="#"`" gap noted in the review).

Effort: Medium (library primitive Small; schema + dual-read + migration the bulk) · **Risk:** medium — password paths are sensitive; land behind the CI gate (Phase 0) with unit tests for create/verify/rehash and a golden-vector test proving legacy verification still works during the window.

5. Sequenced roadmap

Seq	Item	Sev	Effort	Depends on	Category lift
1 ⚠️	H-2 externalize secrets, least-priv login, drop <code>TrustServerCertificate</code>	High	S–M	—	Security
2	H-3 git init + CI (build/test/secret-scan/SAST)	High	M	—	Deployment, Testability
3 ✅	C-1 fail-closed guard (never succeed on an unverified credential)	Critical	S	—	Security
4 ✅	H-4 guard-ordering assertion	High	S	—	Security
5 ✅	Security headers (CSP + friends)	Med	S–M	—	Security
6 ✅	C-2 <code>PasswordHash</code> in <code>Tf1SecurityCrypto</code> + rehash-on-login	Critical	M	—	Security
7 ✅	H-1 global antiforgery + fetch-endpoint token	High	S–M	—	Security
8 ✅	Login-specific rate limiting on all hosts (<code>cbs-login</code> policy in the framework)	Med	S	—	Security
9 🕒	H-6 optimistic concurrency + verify <code>X_SCROLL</code> constraint	High	M	—	Database, Testability

(✅ = done · ⚠️ = partial · unmarked = remaining. Note: the actual execution order was C-1 → C-2 → H-2 → H-1; the CI safety net (H-3) was deferred rather than done first, so the earlier "depends on H-3" gating was not applied.)

Rough horizon: for a small focused team, clearing the gate is on the order of **4–6 weeks**, with the mass concentrated in C-2 (password migration + tests), H-3 (CI, incl. a containerized SQL for real test runs), and H-6 (schema + every write service). The Phase-1 items (C-1, H-4, headers, H-2) are collectively a few days.

6. Definition of done (per item)

- **C-1** — a wrong password with `MFA_AD_LOGIN_ENABLED` (or the QR/OTP flags) set to any value **fails** — the flag-on-but-no-authenticator path returns `Result.Fail`, never success; a test pins the invariant (*wrong password + flag on ⇒ fails*). AD/QR/OTP behavior is decided: on the roadmap, tracked in [BACKLOG.md](#), out of this gate.
- **C-2** — new logins store PBKDF2 versioned hashes; legacy users transparently upgrade on login; no plaintext/reversible value is ever written; unit + golden-vector tests green in CI.
- **H-1** — a forged cross-site POST (Save/Login/Logout) is rejected; the fetch search endpoints send and validate the token.

- **H-2** — no connection string, password, or key in committed config; app runs under a least-privilege SQL login; `TrustServerCertificate` gone; secret scan clean in CI.
- **H-3** — CI blocks a PR that fails build, tests, arch tests, secret scan, or SAST.
- **H-4** — mis-ordering the middleware fails fast at startup (or the guard stays fail-closed on a null endpoint for a routed path), proven by a test.
- **H-6** — two concurrent edits to the same record produce a clean `Result.Fail` on the second, not a silent overwrite; `X_SCROLL` unique constraint confirmed present per provider. (🔗 *Reference implementation landed on the **State master** using **TfIOmniDb's native row-version** — mark the column `[DbColumn(IsRowVersion=true)]` and the repository auto-increments it, appends the `AND VERSION=@expected` guard, and throws `ConcurrencyException` on a stale update; a passing stale-version conflict test on the dev DB proves it. The remaining write services adopt the same pattern — rollout order deferred.*)
- **Login rate limiting (M-3)** ✅ — credential POSTs beyond the budget answer **429 + Retry-After** on **every** host, not just behind the gateway, and the permitted attempts still reach the pipeline (a limiter that rejected from the first request would satisfy a naive "eventually 429" check while breaking sign-in). Proven by tests that boot the real host: the budget trips, each credential endpoint has its own, `Enabled=false` leaves login working rather than 500ing, and the pinned limit is asserted to actually reach the framework. Trust prerequisite included: a backend behind the gateway resolves the **real client** IP (else the partition is one shared bucket); a malformed `ForwardedHeaders` trust entry fails startup instead of being skipped; and a **missing** one announces itself — a discarded `X-Forwarded-For` logs one `Security` warning per host start, so "the limiter is running but throttling everyone as one client" is visible in the log rather than inferred from a wrong `returnUrl`.
- **Headers** ✅ — response carries a fully strict CSP (`script-src / style-src 'self'` , no nonce, no `unsafe-inline`) + `X-Frame-Options/X-Content-Type-Options/Referrer-Policy/Permissions-Policy` on every response incl. static files; the app renders with **zero CSP violations** (achieved by externalising all inline scripts, moving inline handlers to delegated `data-cbs-*` listeners, and inline styles to CSS classes — no nonce needed).

7. Risks & watch-items

- `MFA_AD_LOGIN_ENABLED` **semantics** — **RESOLVED (2026-07-10)**. AD (and QR/OTP) login is confirmed on the roadmap, so C-1 does **not** delete the branch; it makes the path **fail-closed** until a real authenticator exists, and the login modes become backlog features (§8, [BACKLOG.md](#)). Remaining discipline: never let the flag's presence imply a trusted session, and don't ship the AD stub returning success. (*This unblocks C-1 and C-2 — both previously waited on this decision.*)
- `X_SCROLL` **unique constraint (blocks part of H-6)**. The existing MAX+1 duplicate-key retry only works if the DB enforces the unique index. Verify it exists in SQL Server, Oracle, and PostgreSQL targets before relying on it.
- **Password migration window**. Legacy verification must keep working until all active users have logged in once. Keep the dual-read path until a measured cutover; force-reset dormant accounts rather than carrying legacy verification forever.
- **CSP breakage** — **RESOLVED (2026-07-18)**. Rather than nonce the inline surface, it was removed: 3 inline `<script>` blocks externalised to `.js` , 15 inline `on=` handlers (incl. `_Notification.cshtml`) moved to delegated `data-cbs-` listeners in `site.js` , and 32 inline `style=` attributes moved to CSS classes. The one JSON `<script>` island is `type="application/json"` (non-executable — CSP never blocks it), so **no nonce was needed**. Shipped as a fully strict enforced CSP with zero console violations.
- **Secret rotation**. Externalizing secrets (H-2) is also the moment to *rotate* the exposed `sa / sql@2019` credentials — they must be considered compromised since they were in plaintext config.

- **Proxy trust on the IIS farm (open, manual).** Several controls silently depend on each backend trusting the gateway's `X-Forwarded-For`: the M-3 login limiter's per-client partition, `b_LoginPolicy.RemoteHostLoginAttempt`'s per-IP lockout, the same-workstation force-logout, and request attribution in the logs. Untrusted, all of them see one client — the gateway. The Docker stacks now configure this themselves; **an IIS farm still sets `ForwardedHeaders:KnownProxies` by hand per site**. Survival across a redeploy depends on *where* it was put: the documented route ([iis-multi-host.md](#) §3) is each site's `appsettings.Production.json`, which `dotnet publish` leaves alone (no host project ships that file, and publish doesn't clean the output folder) — but anything placed in `web.config <environmentVariables>` **is** wiped, because ANCM regenerates `web.config` on every publish. `deploy-cbs-iis.ps1` prints the reminder and the consequences. **Mitigated 2026-07-31 (detection, not enforcement):** `UseCbsForwardedHeaders` now logs one `Security`-category warning per host start when an `X-Forwarded-For` arrives and is **discarded** because the peer isn't trusted — detected by the header still standing with no `X-Original-For`, the marker the framework middleware writes for what it consumes. Silent when configured correctly and when no proxy exists, so it is a signal rather than noise; it names both the fix and what is silently degraded meanwhile. That converts the whole class from invisible to greppable on every deployment shape. Still not *enforced* — a host with the wrong trust list serves happily — so the post-deploy check stands: an unauthenticated redirect's `returnUrl` must carry the **gateway** origin, not a backend port.
-

8. Explicitly out of scope (the next band, 80+)

Not part of this gate — listed so the boundary is clear.

Login-mode features (confirmed roadmap — see [BACKLOG.md](#) "Authentication & login modes"). The full **AD**, **QR-at-login**, and **OTP-at-login** modes (plus the legacy **RC4** login variant) are feature work, *not* gate items. C-1's gate scope is only the fail-closed guard that stops the bypass; these implementations follow on their own timeline. The guard must be in place *before or with* the first mode, and each mode must authenticate via its own mechanism before the local password compare is skipped — none may bypass verification. AD in particular carries an infra decision (LDAP bind vs. Kerberos/integrated Windows vs. an OIDC broker) that interacts with the gateway/multi-host topology and deserves its own short design note.

Score-lifting items (~74 → ~85). OpenTelemetry traces + metrics + APM (Observability 4→8); distributed session (Redis) + gateway affinity (Scalability 4→7); DB-side filter/sort/page on all providers + reference-data caching + Polly transient-retry (Performance 5→8); ~~DB-migration tooling — DbUp/Flyway~~ (done — `Tf1Cbs.Tools.DbMigrator`; Database 5→6, and 8 once a `0000` legacy-schema baseline exists); extract a `*.Abstractions` assembly + fix the bare-DLL Debug-path references (Dependency Mgmt 6→8); ~~controller + Playwright E2E tests~~ (**done 2026-07-29/30**; Testability 5→7 — the remaining 7→8 is making the DAL mockable, not more tests); container hardening, DR/HA design. Each is real work with a clear payoff, but none blocks a defensible go-live once §5 is green.

9. The road to 100 — remaining work by category

Everything still open (from the ~74 gate all the way to 100), grouped by **who owns it**: **Coding** = application code in this repo · **Infra** = platform/pipeline/hosting (DevOps + ops) · **Other** = DBA credential/cert work + one-off security-process engagements. "Shared" straddles two. **+pts = tentative overall-score gain** on completing that item — a **holistic**

estimate for prioritisation, not an exact additive sum (ship-blockers lift the number more than their own cell).

Progress states how far each item actually is — ☒ done ·  partial (what landed, what is left) · ☐ not started — so a partially-collected item is never read as finished. (Current score: ~77 — Observability instrumentation + the H-6 State pilot took it to ~74; #12 DB migration tooling took it to ~75; the controller/HTTP + browser test tiers took it to ~76; M-4 PII masking took it to ~76.5; M-3 login rate limiting took it to ~77.)

2026-07-31 — M-3 (login rate limiting) closed on every host: score ~76.5 → ~77. The ~90-band row "Login-specific rate limiting on all hosts (not just the gateway)" is ☒ **done**, and as with M-4 the finding was **two gaps stated as one**. The headline half — nothing on a directly-reached host, so the single-host deployments had no limiter at all — is the obvious one. The quieter half is that the limiter that *did* exist was **volumetric, not credential-specific**: 600 requests / 60 s per IP across all traffic allows roughly 600 password guesses a minute, so even the multi-host shape had no working brute-force control, only the appearance of one. **Fix**: a `cbs-login` policy registered in `AddCbsFramework` with `UseRateLimiter()` inserted **first** in `UseCbsFramework()`, ahead of session load, identity-cookie decryption and any DB round-trip — the cheapest point at which an attempt can be refused. The two credential-accepting POSTs opt in with `[EnableRateLimiting]`, and because `AccountController` lives in `Tf1Cbs.Core.Authentication.Web` — an application part in **all five hosts** — one attribute covers every deployment shape instead of five copies that drift apart. Partition is **client IP + request path** (10 / 60 s by default), so a run against the login form cannot also deny the anonymous change-password screen. Three calls worth recording because each rejects an obvious-looking alternative: the **per-username** dimension stays with the DB lockout, since partitioning on the posted username requires a synchronous form-body read that Kestrel disables by default; the 429 carries **no body**, because the middleware owns the status code around `OnRejected` and writing one risks starting the response first; and `Enabled=false` still **registers** the policy as a no-limiter, because an unregistered policy name is a request-time 500 on the login page — a config flag must not be able to break sign-in. A zero or negative size is a **startup error**, never a silent "off". **The prerequisite closed with it, and was the larger piece of work**. The partition only means anything if the gateway's `X-Forwarded-For` is trusted — and `compose.multi.yaml` shipped with trust commented out, not from neglect but because `UseCbsForwardedHeaders` read `KnownProxies` (exact IPs) while a Compose service's IP changes on every `up`. Untrusted, every backend sees the gateway as the client and three controls collapse together: the limiter throttles the bank as one bucket, `RemoteHostLoginAttempt` counts all users' failures against one row, and request logs credit the gateway. So the framework gained `CIDR KnownNetworks`, the plain stack trusts the bridge pool, the HTTPS overlay tightens back to the single static gateway IP, and **malformed trust entries now fail startup** rather than being skipped — the previous `TryParse -and-ignore` let a typo read as a trust list that wasn't in force. And because the *missing*-entry case throws nothing and breaks no page, `UseCbsForwardedHeaders` also gained a **detector**: one `Security` warning per host start when an `X-Forwarded-For` is discarded as untrusted, identified exactly (the header still standing with no `X-Original-For`, the marker the framework middleware writes for what it consumed) rather than guessed at. **25 tests** in total; seven boot the real host deliberately, because the parser tests would all still pass under a renamed config key or a dropped detector, and the production symptom of either is silence, not a crash. **Not claimed**: IIS farms still pin `KnownProxies` by hand (see §7), and the gateway keeps its volumetric limiter as a separate edge layer. **2026-07-31 — M-4 (PII masking) closed on every host: score ~76 → ~76.5.** The ~90-band row "PII masking shared to thin hosts" is ☒ **done**, and the original finding **under-stated the defect**. It read "masking would not apply on those hosts"; the real behaviour was quieter and worse to detect — `MaskingOptions` defaults `keys` to the banking PII/secret list but defaults `Patterns` to **empty**, so the four thin hosts ran **half-masked**: property-name redaction worked, value-regex redaction (a PAN or Aadhaar sitting inside a free-text message) silently did not. Nothing threw, nothing logged, and the review's own strength claim — "recursive PII/secret masking ... wired into every host" — read as true, because the **enricher** was wired everywhere; only its **config** was not. **Fix**: the block was extracted to a canonical `Tf1Cbs.Host.Main/logmasking.json` and **deleted** from `appsettings.json` (leaving it would have kept the copy that drifts — the bug one level up), linked into all four thin

hosts exactly like `routecutover.json`, and loaded in each `Program.cs` **before** `ConfigureTfLogging()` from `AppContext.BaseDirectory` — a linked file is absent from the content root under `dotnet run`. Two deliberate calls beyond the plan: **optional: false**, because a missing file degrades to half-masked logging with nothing to notice, so the host now refuses to start; and `AddEnvironmentVariables()` **re-added afterwards**, because `AddJsonFile` appends a provider and would otherwise shadow `TfLog__Masking__` set in `compose` or an `IIS web.config <environmentVariables>` block. `LogMaskingConfigTests` holds the shape (canonical file defines `Keys` **and** non-empty `Patterns`; every `Sdk.Web` host that builds a `TfOmniLog` pipeline ships it, loads it non- optionally before the pipeline, and re-adds the env provider; no host `appsettings.json` re-declares the section) and was **verified by mutation** — both regressions reintroduced, both caught. **The full +0.5 is taken**, unlike the test row's discount: this fix ships with the build whether or not CI runs anything; only the guard is what H-3 still leaves unexecuted. **Not claimed: M-5** (`Database:LogParameterValues` can log parameter values) is untouched, and no test asserts end-to-end that a PAN written through a real thin-host log file comes out redacted — the enricher and the config delivery are each covered, their join is not. **2026-07-30 — controller + browser test tiers shipped: Testability 5 → 7, score ~75 → ~76.** The ~90-band row "Controller + end-to-end (Playwright) test coverage" is  **done in both halves** — `TfLCbs.IntegrationTests` (23 tests, real pipeline in-process, middleware **order** under test, guard half needs no database) and `TfLCbs.E2E` (80 tests, exit 0, Playwright/TypeScript, **no app source changes**). Reusable-component coverage runs against the Development-gated `/Dev/Components` showcase, so the bank database is never written; real screens self-calibrate against menu rights and **skip with a reason** rather than counting as covered. **Deliberately banked at ~+1, not the row's +1.5:** until H-3 lands there is no CI to run any of it, and a suite nothing executes catches regressions only when someone remembers to look — the "CI actually runs the suite" half of the Testability row in §2 is still outstanding. The browser tier also **found three real defects**, each pinned with `test.fail()` so the suite stays green while the bug stays visible, and all now tracked as risk rows 22–24 in the review's §12: a signed identity cookie whose session has closed produces an **infinite** `/Account/Login` ⇌ `/Modules` **redirect loop** with no route back to the login form (the multi-host SSO shape exactly); `Model.Disabled` is undone at runtime on the account picker — **latent, not live**, since the sole production consumer (`AccHoldingAmount`) only disables it in edit mode where the account is populated and the safe branch runs; and the component showcase wires `clearTargets` on the child so its dependent lookup clears itself. Separately, the work surfaced an **operational hazard worth a fix (row 21)**: a login attempt on an account dormant longer than `b_LoginPolicy.DeactivateDays` **writes** `a_User.STATUS = 3`, deactivating it with no self-service or admin recovery — only a DBA `UPDATE`. Two accounts were deactivated this way here. `/Administration/ConnectedUsers` already exists and is password-gated, so a reactivate action is the natural home. Evidence: [TfLCbs.E2E/README.md](#). **2026-07-30 — account-list conversion landed; score unchanged at ~75, deliberately.** The account picker now runs one server-side join instead of 24 round-trips and is ~2× faster (SQL Server + Oracle verified). It does **not** move the Performance cell, because that cell is docked for "load-all patterns, no caching, no retry"* and **all three are still true** — the load-all remains, nothing is cached, there is no retry. Recording it as a real improvement that earns no cell movement, rather than inflating the number: the other two thirds of this row are now both **explicitly parked, not forgotten** (2026-07-30). **Reference caching:** the infrastructure is built and production-proven (`IReferenceCache` — distributed-backed, fail-open, inspector + eviction UI; only `BankVars` consumes it), so applying it is wiring — **which masters to cache is a deferred decision, not an engineering gap**; the measured per-request offenders are catalogued in `BACKLOG.md`. **Polly transient-retry: backlogged** with the seam settled (one policy at `TfOmniDb's CommandLogger.ExecuteAsync`, covering all providers), **gated on write-idempotency classification** — in a core banking system a blindly-retried write can double-post. Together they would carry Performance 5→7. The paging third is blocked on a DBA index, with measurements on file.

Item	Progress	Category	Band	+pts (est.)	Effort
H-6 optimistic concurrency (rowversion + guarded UPDATE across write services)	 StateService DONE — the reference implementation; every other write service must now follow it. State carries <code>G_STATE.VERSION</code> as <code>[DbColumn(IsRowVersion = true)]</code>, so TflOmniDb auto-increments it in the SET clause, appends <code>AND VERSION = @expected</code> to the WHERE, and throws <code>ConcurrencyException</code> on 0 rows; <code>StateService.UpdateStateAsync</code> catches that and disambiguates "changed by another user" from "already deleted". Proven on the dev DB by a passing conflict theory, and the signature did not change, so there was no controller/view/Demo churn. The rollout is a repeat of three mechanical steps per service — (1) add the <code>VERSION</code> column via a migration, (2) mark it <code>IsRowVersion</code> on the entity in a hand-authored partial, (3) call <code>UpdateAsync(entity, cols)</code> and catch <code>ConcurrencyException</code>. Until that is done, every write path except State is still last-writer-wins.	Coding	~74 gate	+2	M
H-3 — git init + CI pipeline (build/test/arch/secret-scan/SAST)	☐ Not started. The single largest remaining gate item — the working tree is still not a repository.	Infra	~74 gate	+2	M
H-2 finish — rotate <code>sa</code> , least-privilege SQL login, real TLS cert (drop <code>TrustServerCertificate</code>)	 Partial. Secrets are out of committed config; the three remaining actions are DBA/ops, not code.	Other (DBA/ops)	~74 gate	+1	S–M
Distributed session (Redis) + gateway affinity	☐ Not started. <code>CbsSessionStore</code> is still single-node in-memory.	Coding + Infra	~80	+1.5	M–L
Observability instrumentation — OTel traces/metrics + OTLP + log-trace correlation, all 5 hosts + gateway (<code>AddCbsObservability</code>)	☒ Instrumentation done (gateway→host→DB proven). ☐ Prod backend / dashboards / SLO alerting still to stand up — ops, not code.	Coding + Infra	~80	+1 done · +0.5 ops	M

Item	Progress	Category	Band	+pts (est.)	Effort
Performance — DB-side filter/sort/page on large lists — see docs/perf/AccountList-baseline-2026-07-30.md	 One third done. Account picker converted to one server-side join: 24→1 round-trips, ~2× faster, SQL Server + Oracle verified. DB-side paging measured as a 3.6–11× regression (paged join full-scans G_GLOBALCLIENT , 166k reads for 15 rows) → blocked on B_ACCOUNT(ORGELEMENTID, ACCOUNTSTATUSCODE, ACCOUNTNUMBERFORDISPLAY) , i.e. DBA not Coding . Of the other two thirds: reference caching — infra is built (IReferenceCache : distributed, fail-open, proven by BankVars), scope deferred by decision; Polly retry — backlogged , seam settled at TflOmniDb's CommandLogger.ExecuteAsync , gated on write-idempotency classification. Both tracked in BACKLOG.md .	Coding (paging: Other/DBA)	~80	+0.5 done · +1.0 left	M
DB migration tooling + versioned DDL — Tf1Cbs.Tools.DbMigrator (DbUp), per-provider numbered scripts, CbsSchemaVersions journal, deploy-script drift check, Docker migrate profile	 Done , verified live on SQL Server and Oracle. ☐ Remaining: a 0000 legacy-schema baseline (so an empty DB is reproducible) and a live PostgreSQL run.	Coding + Infra	~80	+1 done	M
~~Controller + end-to-end (Playwright) test coverage~~	 DONE (both halves). Tf1Cbs.IntegrationTests (2026-07-29, 23 tests, ~2s) boots the real Program via WebApplicationFactory , so middleware order is under test; its guard/antiforgery half needs no database. Tf1Cbs.E2E (2026-07-30, 80 tests, exit 0) drives a real browser — Playwright/TypeScript, outside the .slnx , no app source changes . Components are covered against the Development-gated /Dev/Components showcase (canned data, TempData-only notifications), so the bank database is never	Coding	~90	~~+1.5~~ banked	M

Item	Progress	Category	Band	+pts (est.)	Effort
	written; real screens self-calibrate against menu rights and skip with a reason. Found and pinned 3 real defects (<code>Model.Disabled</code> undone at runtime on the account picker; the showcase's dependent picker clearing itself; a signed-cookie/closed-session infinite redirect loop on <code>/Account/Login</code>). Banked as ~+1, not +1.5 — nothing runs these automatically until H-3 (CI) lands.				
Row-token bypass tests + multi-host SSO browser flow (residue of the row above)	☐ Not started — named separately so "test coverage  " does not read as "everything tested." <code>RowToken</code> makes four promises (unforgeable / screen-bound / session-bound / single-use) and none of the negative cases is covered ; the suites only prove a <i>valid</i> token round-trips and that ids never travel as bare integers. Each is a distinct bypass if it regresses. This was blocked at the integration tier (tokens embed the session id, the consumed-nonce set is server-side, and a tokenless POST 400s at antiforgery before the token check) — the browser tier removes that blocker , since a cookie jar and antiforgery round-trip are free in Playwright, making this now the cheapest place to close it. Also open: the gateway login / global-logout flow across the multi-host farm, verified by hand once with nothing pinning it.	Coding	~90	+0.5	S–M
IaC / Kubernetes (repeatable infra)	☐ Not started. Four deploy shapes are scripted but not declarative.	Infra	~90	+1.5	L
DR / HA / backup posture	☐ Not started. No RTO/RPO defined.	Infra	~90	+1	L
Penetration test	☐ Not started (external engagement).	Other (security process)	~90	+1	External

Item	Progress	Category	Band	+pts (est.)	Effort
Container hardening (non-root, HEALTHCHECK, image scan)	☐ Not started. Containers still run as root.	Infra	~90	+0.5	S–M
WAF in front of the app	☐ Not started (procurement decision).	Infra	~90	+0.5	S (procure)
~~PII masking shared to thin hosts~~	✅ DONE (2026-07-31). The finding was under-stated: the thin hosts were half-masked, not unmasked — <code>Patterns</code> defaults to empty, so name-based redaction worked while value-regex redaction (PAN/Aadhaar in free text) silently did not, on the four hosts serving the domain screens. Config extracted to a canonical <code>Tf1Cbs.Host.Main/logmasking.json</code> (deleted from <code>appsettings.json</code> — no second copy to drift), linked into all four thin hosts like <code>route takeover.json</code>, loaded from <code>AppContext.BaseDirectory</code> before <code>ConfigureTf1Logging()</code> and optional: false so a missing file stops the host rather than silently degrading it, with <code>AddEnvironmentVariables()</code> re-added after so <code>Tf1Log__Masking__*</code> still overrides. Pinned by <code>LogMaskingConfigTests</code> (4 facts, mutation-verified). Full +0.5 taken — the fix ships with the build regardless of CI. ☐ Not covered: M-5 (<code>Database:LogParameterValues</code>) and an end-to-end assertion on a real log file.	Coding	~90	+0.5 done	S
~~Login-specific rate limiting on all hosts (not just the gateway)~~	✅ DONE (2026-07-31). Two gaps, both closed by one framework control: the gateway's limiter was <i>volumetric</i> (600/60 s over all traffic ⇒ ~600 guesses/min) and missing from any directly-reached host, including the entire single-host shape. <code>AddCbsFramework</code> registers <code>cbs-login</code>; <code>UseCbsFramework</code> runs <code>UseRateLimiter()</code> first; both credential POSTs opt in. Partition =	Coding	~90	+0.5 done	S

Item	Progress	Category	Band	+pts (est.)	Effort
	client IP + path; default 10/60 s; 429 + Retry-After ; Security log category. Disabling still registers a no-limiter policy (an unknown policy name would 500 the login page); a 0/negative value is a startup error, not a silent "off". 9 tests. Per-user partitioning behind the gateway was the one prerequisite and is also done — ForwardedHeaders gained CIDR KnownNetworks , the Docker multi-host stack now trusts the gateway, malformed trust entries fail startup (12 more tests). ☐ Per-username stays with the DB lockout; IIS farms still pin KnownProxies by hand.				
Threat model	☐ Not started (workshop).	Other (security process)	~90	+0.5	S (workshop)
HSTS strengthening (1-yr max-age + includeSubDomains / preload)	☐ Not started — deliberately deferred; preload is hard to reverse.	Other (ops decision)	backlog	+0.5	S
Migration completeness — port the remaining ~98% of screens	 ~2% ported. 12 domain screens / 47 views across 8 promoted domains — the vertical slice is proven; the volume is not.	Coding	~100	+9	XL

Band subtotals (ladder from the original ~72 baseline; now at ~77): gate items +5 → ~76–77 · ~80-band +5.5 → ~81–82 · ~90-band +5.5 → ~89–90 · migration completeness +9 → ~99–100. (Totals ~+25 remaining, on top of the ~+2 already banked — ~+1 for the test row, +0.5 for PII masking, +0.5 for login rate limiting; directional.)

How to prioritise: best **points-per-effort** are the two gate items — **H-6 (pure Coding)** and **H-3 (pure Infra)** — cheap, high-value, parallelisable; do them first. The ~80-band shared items (Redis, OTel, DB-side paging) are the next-best ratio. The ~90 band is many small-to-medium items with diminishing per-item returns (mostly Infra + security process). The final **+9 to 100 is the migration itself** — by far the largest effort and unavoidable, but pure volume, not architecture (nothing there needs a redesign; structure already scores 8/10).

Observability (OTel) — what the +1.5 buys, and the shape of the work. *Update (2026-07-18 — instrumentation DONE): the app-side instrumentation described below has since landed as AddCbsObservability (OTel traces + metrics + OTLP exporter + the TfIOmniDb DAL ActivitySource + log-trace correlation), wired in all 5 hosts and the gateway, off by default. The **coding** half is complete; what remains is the **Infra/ops backend** (Collector → Prometheus/Tempo + Grafana or a SaaS APM, with retention, dashboards and SLO alerting). The rest of this note is the original rationale. Of the three pillars (logs / metrics / traces), only **logs** were present (TfLog : structured, correlation Reference , categories incl. Performance/Request, per-request middleware). **Metrics and traces are missing** — no request rate / p95–p99 latency / error rate, and no gateway→host→DB span tree — which is the "blind in production" gap: you learn of problems from*

user complaints, and there's nothing to alert on. OpenTelemetry is **vendor-neutral** — instrument once (**turnkey auto-instrumentation** for ASP.NET Core + `HttpClient`, and **DB spans free for SQL Server via `Microsoft.Data.SqlClient` and for PostgreSQL via Npgsql's built-in OTel**), export to any backend. (*Oracle's managed provider emits no OTel spans; and because **TfLOmniDb is editable in-house source** — not a bare DLL — uniform DB spans across all three providers, with the SQL/provider/row-count/correlation-id the DAL already tracks, come from adding a **single `ActivitySource` at the DAL's existing central command seam** — one place, not per-call-site.*) High-value 80/20: **RED metrics** (rate/errors/duration per endpoint) + auto-traces, with **W3C `traceparent` propagated across the YARP gateway** so a request is a single trace, and the OTel trace-id **unified with the existing correlation `Reference`** so a log line links to its trace. The real cost is **Infra/ops, not code**: standing up and operating the backend — OTel Collector → Prometheus/Tempo + Grafana (**self-hosted**, for data-residency) or **SaaS** (App Insights/Datadog) — with retention, dashboards and SLO alerting. Post-gate Medium (the app runs; logs give forensics) — best paired with the Redis stood up for distributed session and with H-3's pipeline. **Decision that drives it: self-hosted vs SaaS backend (data-residency + ops capacity).**

This is a plan, not an implementation. It changes no code. The two Critical items (C-1, C-2) come first — C-1 as a fail-closed guard (the AD/QR/OTP login modes are decided as roadmap features, tracked in [BACKLOG.md](#), not part of this gate); everything else sequences behind the CI safety net so the fixes can't silently regress.